In a sense, the class keyword in Ruby is more like a scope operator than a class declaration. Yes, it creates classes that don't yet exist, but you might argue that it does this as a pleasant side effect. For class, the core job is to move you in the context of the class, where you can define methods.

This distinction about the class keyword is not an academic detail. It has an important practical consequence: you can always reopen existing classes—even standard library classes such as String or Array—and modify them on the fly. You can call this technique *Open Class*.

Spell: Open Class

To see how people use Open Classes in practice, let's look at a quick example from a real-life library.

The Money Example

You can find an example of Open Classes in the money gem, a set of utility classes for managing money and currencies. Here's how you create a Money object:

object_model/money_example.rb
```
require "money"

bargain_price = Money.from_numeric(99, "USD")
bargain_price.format # => "$99.00"
```

As a shortcut, you can also convert any number to a Money object by calling Numeric#to_money:

object_model/money_example.rb
```
require "money"

standard_price = 100.to_money("USD")
standard_price.format # => "$100.00"
```

Since Numeric is a standard Ruby class, you might wonder where the method Numeric#to_money comes from. Look through the source of the Money gem, and you'll find code that reopens Numeric and defines that method:

```
class Numeric
  def to_money(currency = nil)
    Money.from_numeric(self, currency || Money.default_currency)
  end
end
```

It's quite common for libraries to use Open Classes this way.

As cool as they are, however, Open Classes have a dark side—one that you're about to experience.